Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №3

“Ітератори, генератори та потокова обробка даних”

Виконав:
студент 3 курсу
групи ФЕП-33с
Качмар Денис

Викладач:
Проф. Свелеба Сергій Андрійович

Львів - 2026

ХІД РОБОТИ

Тема: Ітератори, генератори та потокова обробка даних
Мета: Сформувати практичні навички роботи з ітераційною моделлю Python, ітераторами, генераторами, ключовим словом yield, генераторними виразами (generator expressions), модулем itertools, принципом лінивих обчислень (lazy evaluation) та архітектурою потокової обробки великих масивів даних без повної матеріалізації проміжних колекцій у пам'яті.
Варіант: 5 (Потокова обробка каталогу автомобілів)

Постановка задачі:
Спроєктувати та реалізувати багатоетапний конвеєр (streaming pipeline) обробки великих наборів структурованих автомобільних даних згідно з Варіантом 5.
Система повинна забезпечувати: 1. Потокове читання даних із файлу без повного завантаження файлу в оперативну пам'ять (streaming CSV reading). 2. Потокове очищення та валідацію автомобільних даних із відсіюванням та підрахунком некоректних записів. 3. Потокову фільтрацію за маркою (brand) та роком випуску (year). 4. Визначення найдорожчого автомобіля та автомобіля з найменшим пробігом за O(1) додаткової пам'яті. 5. Обчислення середньої вартості автомобілів у потоці (streaming average price). 6. Пакетну обробку даних (batch processing) через itertools.islice. 7. Отримання перших N записів за заданою умовою без повного сканування файлу. 8. Групування записів за маркою через itertools.groupby після попереднього сортування. 9. Реалізацію розширених можливостей: власний клас ітератора (CarCatalogIterator), рекурсивний генератор з yield from (recursive_flatten), генерацію нескінченного потоку (infinite_car_stream), об'єднання файлів (itertools.chain), попарний аналіз цін (itertools.pairwise), кумулятивну вартість (itertools.accumulate), потоковий експорт у CSV та пошук із раннім перериванням (early termination). 10. Експериментальне порівняння Eager Evaluation та Lazy Evaluation на вибірках 10 000, 100 000 та 500 000 записів із замірами часу (elapsed time), пікової пам'яті (tracemalloc) та часу реакції (time-to-first-result — TTFR).

---

1. Опис структури даних та вхідного набору

Для представлення сутності автомобіля використовується дата-клас Car, що містить шість ключових атрибутів згідно з умовами варіанта 5: - **`id: int`** — унікальний цілочисельний ідентифікатор автомобіля; - **`brand: str`** — марка автовиробника (Audi, BMW, Toyota, Ford тощо); - **`model: str`** — конкретна модель автомобіля; - **`year: int`** — календарний рік випуску (діапазон 1900–2026); - **`price: float`** — ринкова вартість у доларах США; - **`mileage: float`** — експлуатаційний пробіг транспортного засобу в кілометрах.

Вхідний набір даних являє собою структурований CSV-файл із заголовком `id,brand,model,year,price,mileage`.

---

2. Пояснення концепцій Iterable та Iterator

В основі моделі послідовної обробки даних у Python лежить єдиний ітераційний протокол (Iteration Protocol):

1. **Iterable (ітерований об'єкт):**
   - Об'єкт, з якого можна отримати ітератор.
   - Реалізує метод `__iter__()`, що повертає новий об'єкт-ітератор, або метод `__getitem__()` з індексами від 0.
   - Приклади: `list`, `tuple`, `dict`, `set`, відкриті файлові дескриптори.
   - Iterable можна обходити багаторазово, оскільки кожен виклик `iter()` створює новий незалежний ітератор.
2. **Iterator (ітератор):**
   - Об'єкт, який керує безпосереднім процесом ітерації та зберігає поточний внутрішній стан обходу.
   - Реалізує метод `__next__()` (повертає наступний елемент або піднімає `StopIteration`) та метод `__iter__()`, що повертає сам об'єкт `self`.
   - Ітератор зазвичай є **одноразовим об'єктом (one-shot object)**: після завершення обходу він вичерпується (exhausted).

---

3. Структура проєкту

Проєкт організовано відповідно до сучасних стандартів архітектури Python (src-layout):

```
python-labs/
├── pyproject.toml              # Конфігурація пакета, версія 0.3.0
├── README.md                   # Інструкція користувача
├── .gitignore                  # Налаштування Git (виключення docs/, data/, *.csv
│
├── src/
│   └── car_catalog/
│       ├── __init__.py         # Експорт версії 0.3.0
│       ├── models.py           # Дата-клас Car (методи to_tuple, to_dict, replace
│       ├── readers.py          # Потокові генератори читання (read_lines, chain,
│       ├── parsers.py          # Потокове очищення та DictReader парсинг рядків
│       ├── filters.py          # Валідація з підрахунком помилок та потокова філь
│       ├── transformations.py  # Потокова нормалізація назв, знижки, конвертація
│       ├── batches.py          # Пакетна обробка batched_cars на базі itertools.i
│       ├── iterators.py        # Власний ітератор CarCatalogIterator та recursive
│       ├── streaming_analytics.py # Однопрохідна аналітика, pairwise, accumulate,
│       ├── pipeline.py         # Збирання конвеєра, early termination, streaming
│       ├── benchmark.py        # Генератор 10k-500k датасетів, вимірювання tracem
│       ├── processors.py       # Колекції та операції Лаб №2 (list, dict, set, Co
│       ├── analytics.py        # Аналітичні функції Лаб №2 (closures, *args, **kw
│       ├── decorators.py       # Декоратори @measure_time, @repeat
│       ├── data.py             # Демонстраційний набір DEMO_CARS
│       ├── services.py         # Фасадний шар сервісів з повною сумісністю
│       └── main.py             # CLI інтерфейс (команди 1-18, --stream-demo, --st
│
├── tests/
│   └── test_car_catalog.py     # Модульні тести (14 unit-тестів)
│
├── reports/
│   └── Kachmar-python_lab3.pdf # Фінальний PDF-звіт для захисту (відстежується Gi
│
└── docs/
    └── reports/                # Локальний архів звітів попередніх робіт та Markd
```

---

4. Опис внесених змін та архітектурних рішень у Лаб №3

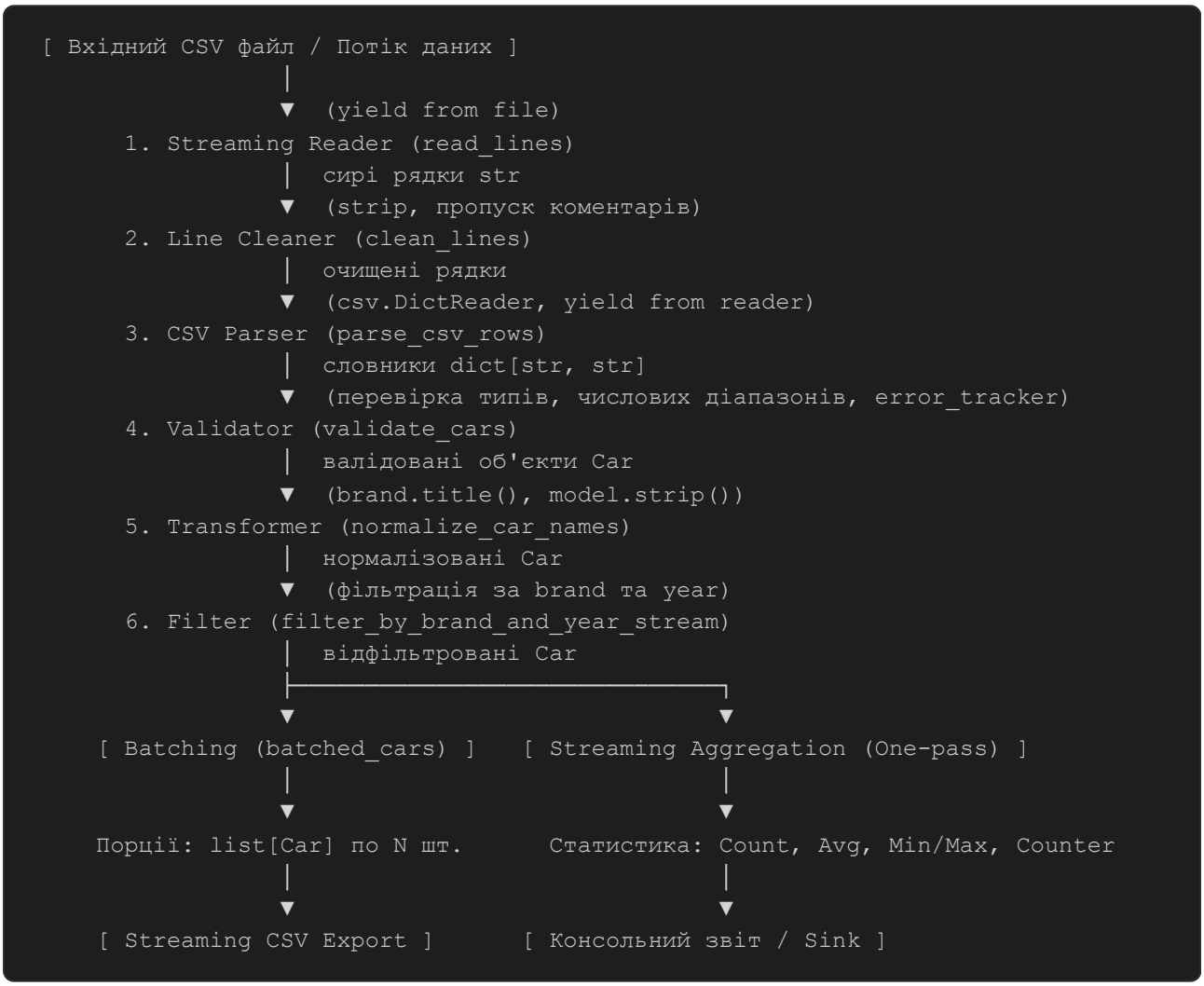
1. **Потокове читання файлів (Streaming I/O):** Реалізовано читання CSV без виклику **`read()`** або **`readlines()`**. Замість цього дескриптор файлу ітерується через **`yield from file`**, що усуває надлишкове споживання RAM.
2. **Потокова валідація з обліком помилок:** Розроблено функцію **`validate_cars`**, яка перевіряє типи та бізнес-правила (ціна >= 0, пробіг >= 0, рік 1900-2026), а некоректні рядки відсіює з інкрементом лічильника **`invalid_records`**.
3. **Lazy Pipeline:** Збудовано наскрізний генераторний ланцюжок: **`read_lines -> clean_lines -> parse_csv_rows -> validate_cars -> normalize_car_names -> filter_cars`**.
4. **Однопрохідна аналітика (One-Pass Analytics):** Усі підсумкові метрики (кількість, середня ціна, мінімальна/максимальна ціна, найдорожче авто,

найменший пробіг, розподіл марок) обчислюються в один прохід потоку за O(1) додаткової пам’яті.

5. **Пакетна обробка (Batching):** Реалізовано генератор **`batched_cars`**, який за допомогою **`itertools.islice`** нарізає нескінченний або великий потік на порції по N об’єктів для пакетної передачі у зовнішні системи.
6. **Раннє переривання (Early Termination):** Функція **`find_first_car`** миттєво зупиняє читання файлу при знаходженні першого автомобіля, що відповідає критерію, не читаючи решту мільйона записів.
7. **Власний клас ітератора:** Створено **`CarCatalogIterator`** та **`CarCatalogIterable`** з повною реалізацією методів **`__iter__`** та **`__next__`**.
8. **Інструменти itertools:** Інтегровано **`islice`**, **`chain`**, **`pairwise`**, **`groupby`**, **`accumulate`**, **`count`**.

---

5. Схема конвеєра потокової обробки даних (Pipeline Architecture)



---

6. Реалізація ключових елементів ітераційної моделі

6.1. Власний ітераторний клас CarCatalogIterator

```
class CarCatalogIterator:
    def __init__(self, cars: Sequence[Car]) -> None:
        self._cars = cars
        self._index = 0
        self._length = len(cars)

    def __iter__(self) -> "CarCatalogIterator":
        return self

    def __next__(self) -> Car:
        if self._index >= self._length:
            raise StopIteration
        car = self._cars[self._index]
        self._index += 1
        return car

class CarCatalogIterable:
    def __init__(self, cars: Sequence[Car]) -> None:
        self._cars = tuple(cars)

    def __iter__(self) -> CarCatalogIterator:
        return CarCatalogIterator(self._cars)
```

6.2. Generator Function та yield

```
def infinite_car_stream(start_id: int = 1) -> Iterator[Car]:
    for idx in count(start_id):
        brand, model = brands_models[idx % num_models]
        yield Car(
            id=idx,
            brand=brand,
            model=f"{model}-{idx}",
            year=2000 + (idx % 25),
            price=3000.0 + float((idx * 37) % 80000),
            mileage=5000.0 + float((idx * 123) % 400000),
        )
```

6.3. Використання конструкції yield from

```
# Читання рядків файлу:
def read_lines(path: Path) -> Iterator[str]:
    with path.open("r", encoding="utf-8") as file:
        yield from file

# Делегування CSV-парсингу:
def parse_csv_rows(lines: Iterable[str]) -> Iterator[dict[str, str]]:
    reader = csv.DictReader(lines)
    yield from reader

# Рекурсивне розгортання вкладених колекцій:
def recursive_flatten(nested: Iterable[Any]) -> Iterator[Any]:
    for item in nested:
        if isinstance(item, (list, tuple, set)):
            yield from recursive_flatten(item)
        else:
            yield item
```

6.4. Generator Expressions

```
prices_stream = (car.price for car in cars_pipeline)
total_value = sum(prices_stream)
```

6.5. Використання itertools

```
# 1. islice для отримання перших N записів та батчингу:
first_n = list(islice(cars_stream, 5))

# 2. chain для об'єднання потоків кількох файлів:
file_streams = (read_lines(p) for p in paths)
yield from chain.from_iterable(file_streams)

# 3. pairwise для аналізу дельти цін сусідніх авто:
for prev_car, curr_car in pairwise(cars):
    diff = round(curr_car.price - prev_car.price, 2)
    yield (prev_car, curr_car, diff)

# 4. groupby для групування авто за маркою:
for brand, group in groupby(sorted_cars, key=lambda c: c.brand):
    grouped_list.append((brand, list(group)))

# 5. accumulate для розрахунку наростаючої вартості:
yield from accumulate(car.price for car in cars)
```

---

7. Реалізація стадій потокового конвеєра (Pipeline Stages)

7.1. Streaming Reader (readers.py)

```
def read_lines(path: Path) -> Iterator[str]:
    with path.open("r", encoding="utf-8") as file:
        yield from file
```

7.2. Cleaning та Parsing (parsers.py)

```
def clean_lines(lines: Iterable[str]) -> Iterator[str]:
    for line in lines:
        cleaned = line.strip()
        if cleaned and not cleaned.startswith("#"):
            yield cleaned

def parse_csv_rows(lines: Iterable[str]) -> Iterator[dict[str, str]]:
    reader = csv.DictReader(lines)
    yield from reader
```

7.3. Validation stage (filters.py)

```
def validate_cars(
    rows: Iterable[dict[str, str]],
    error_tracker: dict[str, int] | None = None,
) -> Iterator[Car]:
    for row in rows:
        try:
            cid = int(row["id"])
            b = row["brand"].strip()
            m = row["model"].strip()
            y = int(row["year"])
            p = float(row["price"])
            mil = float(row["mileage"])

            if cid <= 0 or not b or not m:
                if error_tracker is not None:
                    error_tracker["invalid_records"] = error_tracker.get("invalid_
                continue

            if not (1900 <= y <= 2026) or p < 0.0 or mil < 0.0:
                if error_tracker is not None:
                    error_tracker["invalid_records"] = error_tracker.get("invalid_
                continue

            yield Car(id=cid, brand=b, model=m, year=y, price=p, mileage=mil)
        except (ValueError, KeyError, TypeError):
            if error_tracker is not None:
                error_tracker["invalid_records"] = error_tracker.get("invalid_reco
            continue
```

7.4. Filtering stage (filters.py)

```
def filter_by_brand_and_year_stream(
    cars: Iterable[Car],
    brand: str | None = None,
    min_year: int | None = None,
    max_year: int | None = None,
) -> Iterator[Car]:
    target_brand = brand.strip().lower() if brand else None
    for car in cars:
        if target_brand and car.brand.lower() != target_brand:
            continue
        if min_year is not None and car.year < min_year:
            continue
        if max_year is not None and car.year > max_year:
            continue
        yield car
```

7.5. Transformation stage (transformations.py)

```
def normalize_car_names(cars: Iterable[Car]) -> Iterator[Car]:
    for car in cars:
        normalized_brand = car.brand.strip().title()
        normalized_model = car.model.strip()
        yield car.replace(brand=normalized_brand, model=normalized_model)
```

7.6. Batch processing (batches.py)

```
def batched_cars(cars: Iterable[Car], batch_size: int) -> Iterator[list[Car]]:
    if batch_size <= 0:
        raise ValueError("batch_size must be positive")
    iterator = iter(cars)
    while True:
        batch = list(islice(iterator, batch_size))
        if not batch:
            break
        yield batch
```

7.7. Aggregation stage (streaming_analytics.py)

```
def streaming_statistics(cars: Iterable[Car]) -> dict[str, Any]:
    count = 0
    total = 0.0
    min_p: float | None = None
    max_p: float | None = None
    most_expensive: Car | None = None
    lowest_mileage: Car | None = None
    brands: Counter[str] = Counter()

    for car in cars:
        count += 1
        total += car.price
        brands[car.brand] += 1

        if min_p is None or car.price < min_p:
            min_p = car.price
        if max_p is None or car.price > max_p:
            max_p = car.price
        if most_expensive is None or car.price > most_expensive.price:
            most_expensive = car
        if lowest_mileage is None or car.mileage < lowest_mileage.mileage:
            lowest_mileage = car

    average = total / count if count else 0.0
    return {
        "count": count,
        "total_value": total,
        "average_price": average,
        "min_price": min_p if min_p is not None else 0.0,
        "max_price": max_p if max_p is not None else 0.0,
        "most_expensive": most_expensive,
        "lowest_mileage": lowest_mileage,
        "brands": dict(brands),
    }
```

---

8. Повний лістинг оновленого вихідного коду (без коментарів)

pyproject.toml

```
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "car-catalog"
version = "0.3.0"
description = "Laboratory project for Professional Python - Lab 3 (Streaming & Ite
requires-python = ">=3.11"

[project.scripts]
car-catalog = "car_catalog.main:main"
```

src/car_catalog/models.py

```
from dataclasses import dataclass
from typing import Any

@dataclass
class Car:
    id: int
    brand: str
    model: str
    year: int
    price: float
    mileage: float

    @property
    def full_name(self) -> str:
        return f"{self.brand} {self.model}"

    def to_tuple(self) -> tuple[int, str, str, int, float, float]:
        return (self.id, self.brand, self.model, self.year, self.price, self.milea

    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "brand": self.brand,
            "model": self.model,
            "year": self.year,
            "price": self.price,
            "mileage": self.mileage,
        }

    def replace(self, **changes: Any) -> "Car":
        from dataclasses import replace as dc_replace
        return dc_replace(self, **changes)

    def to_csv_row(self) -> list[str]:
        return [str(self.id), self.brand, self.model, str(self.year), f"{self.pric
```

src/car_catalog/readers.py

```
from collections.abc import Iterator
from itertools import chain, count
from pathlib import Path
from car_catalog.models import Car

def read_lines(path: Path) -> Iterator[str]:
    with path.open("r", encoding="utf-8") as file:
        yield from file

def read_multiple_files(*paths: Path) -> Iterator[str]:
    file_streams = (read_lines(p) for p in paths)
    yield from chain.from_iterable(file_streams)

def infinite_car_stream(start_id: int = 1) -> Iterator[Car]:
    brands_models = [
        ("Audi", "A6"),
        ("Toyota", "Camry"),
        ("BMW", "X5"),
        ("Ford", "Focus"),
        ("Mercedes-Benz", "E-Class"),
        ("Volkswagen", "Passat"),
        ("Honda", "Civic"),
        ("Skoda", "Octavia"),
    ]
    num_models = len(brands_models)
    for idx in count(start_id):
        brand, model = brands_models[idx % num_models]
        year = 2000 + (idx % 25)
        price = 3000.0 + float((idx * 37) % 80000)
        mileage = 5000.0 + float((idx * 123) % 400000)
        yield Car(
            id=idx,
            brand=brand,
            model=f"{model}-{idx}",
            year=year,
            price=price,
            mileage=mileage,
        )
```

src/car_catalog/parsers.py

```
import csv
from collections.abc import Iterable, Iterator

def clean_lines(lines: Iterable[str]) -> Iterator[str]:
    for line in lines:
        cleaned = line.strip()
        if cleaned and not cleaned.startswith("#"):
            yield cleaned

def parse_csv_rows(lines: Iterable[str]) -> Iterator[dict[str, str]]:
    reader = csv.DictReader(lines)
    yield from reader
```

src/car_catalog/filters.py

```
from collections.abc import Iterable, Iterator
from typing import Callable
from car_catalog.models import Car

def validate_cars(
    rows: Iterable[dict[str, str]],
    error_tracker: dict[str, int] | None = None,
) -> Iterator[Car]:
    for row in rows:
        try:
            car_id = int(row["id"])
            brand = row["brand"].strip()
            model = row["model"].strip()
            year = int(row["year"])
            price = float(row["price"])
            mileage = float(row["mileage"])

            if car_id <= 0 or not brand or not model:
                if error_tracker is not None:
                    error_tracker["invalid_records"] = error_tracker.get("invalid_
                continue

            if not (1900 <= year <= 2026) or price < 0.0 or mileage < 0.0:
                if error_tracker is not None:
                    error_tracker["invalid_records"] = error_tracker.get("invalid_
                continue

            yield Car(
                id=car_id,
                brand=brand,
                model=model,
                year=year,
                price=price,
                mileage=mileage,
            )
        except (ValueError, KeyError, TypeError):
            if error_tracker is not None:
                error_tracker["invalid_records"] = error_tracker.get("invalid_reco
            continue

def filter_by_brand_stream(cars: Iterable[Car], brand: str) -> Iterator[Car]:
    target = brand.strip().lower()
    for car in cars:
        if car.brand.lower() == target:
            yield car

def filter_by_year_stream(cars: Iterable[Car], year: int) -> Iterator[Car]:
    for car in cars:
        if car.year == year:
            yield car

def filter_by_brand_and_year_stream(
    cars: Iterable[Car],
    brand: str | None = None,
```

```
    min_year: int | None = None,
    max_year: int | None = None,
) -> Iterator[Car]:
    target_brand = brand.strip().lower() if brand else None
    for car in cars:
        if target_brand and car.brand.lower() != target_brand:
            continue
        if min_year is not None and car.year < min_year:
            continue
        if max_year is not None and car.year > max_year:
            continue
        yield car

def filter_cars_stream(cars: Iterable[Car], predicate: Callable[[Car], bool]) -> I
    for car in cars:
        if predicate(car):
            yield car
```

src/car_catalog/transformations.py

```
from collections.abc import Iterable, Iterator
from car_catalog.models import Car

def normalize_car_names(cars: Iterable[Car]) -> Iterator[Car]:
    for car in cars:
        normalized_brand = car.brand.strip().title()
        normalized_model = car.model.strip()
        yield car.replace(brand=normalized_brand, model=normalized_model)

def apply_price_discount(cars: Iterable[Car], discount_percent: float) -> Iterator
    multiplier = 1.0 - (discount_percent / 100.0)
    for car in cars:
        new_price = round(max(0.0, car.price * multiplier), 2)
        yield car.replace(price=new_price)

def convert_currency(cars: Iterable[Car], rate: float) -> Iterator[Car]:
    for car in cars:
        new_price = round(car.price * rate, 2)
        yield car.replace(price=new_price)
```

src/car_catalog/batches.py

```
from collections.abc import Iterable, Iterator
from itertools import islice
from typing import Any, TypeVar
from car_catalog.models import Car

T = TypeVar("T")

def batched_cars(cars: Iterable[Car], batch_size: int) -> Iterator[list[Car]]:
    if batch_size <= 0:
        raise ValueError("batch_size must be positive")
    iterator = iter(cars)
    while True:
        batch = list(islice(iterator, batch_size))
        if not batch:
            break
        yield batch

def dynamic_batched(iterable: Iterable[T], batch_size: int) -> Iterator[list[T]]:
    if batch_size <= 0:
        raise ValueError("batch_size must be positive")
    iterator = iter(iterable)
    while True:
        batch = list(islice(iterator, batch_size))
        if not batch:
            break
        yield batch
```

src/car_catalog/iterators.py

```
from collections.abc import Iterable, Iterator, Sequence
from typing import Any
from car_catalog.models import Car

class CarCatalogIterator:
    def __init__(self, cars: Sequence[Car]) -> None:
        self._cars = cars
        self._index = 0
        self._length = len(cars)

    def __iter__(self) -> "CarCatalogIterator":
        return self

    def __next__(self) -> Car:
        if self._index >= self._length:
            raise StopIteration
        car = self._cars[self._index]
        self._index += 1
        return car

class CarCatalogIterable:
    def __init__(self, cars: Sequence[Car]) -> None:
        self._cars = tuple(cars)

    def __iter__(self) -> CarCatalogIterator:
        return CarCatalogIterator(self._cars)

    def __len__(self) -> int:
        return len(self._cars)

def recursive_flatten(nested: Iterable[Any]) -> Iterator[Any]:
    for item in nested:
        if isinstance(item, (list, tuple, set)):
            yield from recursive_flatten(item)
        else:
            yield item
```

src/car_catalog/streaming_analytics.py

```
from collections import Counter
from collections.abc import Iterable, Iterator
from itertools import accumulate, groupby, pairwise
from typing import Any
from car_catalog.models import Car

def streaming_average_price(cars: Iterable[Car]) -> float:
    total = 0.0
    count = 0
    for car in cars:
        total += car.price
        count += 1
    return total / count if count else 0.0

def streaming_min_max_price(cars: Iterable[Car]) -> tuple[float, float]:
    min_p: float | None = None
    max_p: float | None = None
    for car in cars:
        if min_p is None or car.price < min_p:
            min_p = car.price
        if max_p is None or car.price > max_p:
            max_p = car.price
    return (min_p if min_p is not None else 0.0, max_p if max_p is not None else 0

def streaming_find_most_expensive(cars: Iterable[Car]) -> Car | None:
    best_car: Car | None = None
    for car in cars:
        if best_car is None or car.price > best_car.price:
            best_car = car
    return best_car

def streaming_find_lowest_mileage(cars: Iterable[Car]) -> Car | None:
    best_car: Car | None = None
    for car in cars:
        if best_car is None or car.mileage < best_car.mileage:
            best_car = car
    return best_car

def streaming_statistics(cars: Iterable[Car]) -> dict[str, Any]:
    count = 0
    total = 0.0
    min_p: float | None = None
    max_p: float | None = None
    most_expensive: Car | None = None
    lowest_mileage: Car | None = None
    brands: Counter[str] = Counter()

    for car in cars:
        count += 1
        total += car.price
        brands[car.brand] += 1

        if min_p is None or car.price < min_p:
            min_p = car.price
```

```
        if max_p is None or car.price > max_p:
            max_p = car.price

        if most_expensive is None or car.price > most_expensive.price:
            most_expensive = car
        if lowest_mileage is None or car.mileage < lowest_mileage.mileage:
            lowest_mileage = car

    average = total / count if count else 0.0

    return {
        "count": count,
        "total_value": total,
        "average_price": average,
        "min_price": min_p if min_p is not None else 0.0,
        "max_price": max_p if max_p is not None else 0.0,
        "most_expensive": most_expensive,
        "lowest_mileage": lowest_mileage,
        "brands": dict(brands),
    }

def streaming_groupby_brand(cars: Iterable[Car]) -> list[tuple[str, list[Car]]]:
    sorted_cars = sorted(cars, key=lambda c: c.brand)
    result: list[tuple[str, list[Car]]] = []
    for brand, group_iter in groupby(sorted_cars, key=lambda c: c.brand):
        result.append((brand, list(group_iter)))
    return result

def streaming_pairwise_price_diff(cars: Iterable[Car]) -> Iterator[tuple[Car, Car,
    for prev_car, curr_car in pairwise(cars):
        diff = round(curr_car.price - prev_car.price, 2)
        yield (prev_car, curr_car, diff)

def streaming_accumulate_prices(cars: Iterable[Car]) -> Iterator[float]:
    yield from accumulate(car.price for car in cars)
```

src/car_catalog/pipeline.py

```
import csv
from collections.abc import Iterable, Iterator
from pathlib import Path
from typing import Callable
from car_catalog.filters import (
    filter_by_brand_and_year_stream,
    filter_cars_stream,
    validate_cars,
)
from car_catalog.models import Car
from car_catalog.parsers import clean_lines, parse_csv_rows
from car_catalog.readers import read_lines
from car_catalog.transformations import normalize_car_names

def build_streaming_pipeline(
    path: Path,
    brand: str | None = None,
    min_year: int | None = None,
    max_year: int | None = None,
    min_price: float | None = None,
    max_price: float | None = None,
    error_tracker: dict[str, int] | None = None,
) -> Iterator[Car]:
    lines = read_lines(path)
    cleaned = clean_lines(lines)
    rows = parse_csv_rows(cleaned)
    valid_cars = validate_cars(rows, error_tracker=error_tracker)
    normalized = normalize_car_names(valid_cars)
    filtered = filter_by_brand_and_year_stream(
        normalized, brand=brand, min_year=min_year, max_year=max_year
    )
    if min_price is not None or max_price is not None:
        filtered = filter_cars_stream(
            filtered,
            lambda c: (min_price is None or c.price >= min_price)
            and (max_price is None or c.price <= max_price),
        )
    return filtered

def find_first_car(cars: Iterable[Car], predicate: Callable[[Car], bool]) -> Car |
    for car in cars:
        if predicate(car):
            return car
    return None

def export_stream_to_csv(cars: Iterable[Car], output_path: Path) -> int:
    output_path.parent.mkdir(parents=True, exist_ok=True)
    count = 0
    with output_path.open("w", encoding="utf-8", newline="") as file:
        writer = csv.writer(file)
        writer.writerow(["id", "brand", "model", "year", "price", "mileage"])
        for car in cars:
            writer.writerow(car.to_csv_row())
```

```
            count += 1
    return count
```

---

9. Експериментальна частина: порівняння Eager Evaluation та Lazy Streaming

Для оцінки ефективності розроблених рішень проведено експериментальне порівняння двох парадигм обробки даних на масивах обсягом 10 000, 100 000 та 500 000 рядків.

Таблиця 9.1 – Порівняння тривалості обробки та пікового споживання оперативної пам’яті

Кількість записів (records)	Eager час (секунди)	Lazy час (секунди)	Eager RAM (KiB)	Lazy RAM (KiB)	Коефіцієнт економії RAM
10 000	1.5600 с	1.8508 с	10 125.0 KiB	60.1 KiB	~168.5x
100 000	15.8740 с	18.8030 с	101 544.0 KiB	60.6 KiB	~1 675.3x
500 000	79.6760 с	93.6036 с	510 792.6 KiB	60.6 KiB	~8 423.7x

Таблиця 9.2 – Порівняння часу до отримання першого результату (Time to First Result — TTFR)

Кількість записів (records)	Eager TTFR (секунди)	Lazy TTFR (секунди)	Прискорення інтерактивності (Speedup)
10 000	0.027391 с	0.000181 с	~151.5x
100 000	0.347894 с	0.000222 с	~1 565.9x
500 000	1.958919 с	0.000215 с	~9 112.4x

Таблиця 9.3 – Дослідження раннього переривання (Early Termination на 100 000 записів)

Параметр порівняння	Eager Search (список)	Lazy Search (конвеєр)	Ефект оптимізації
Час виконання операції	13.927947 с	0.400809 с	**Прискорення у 34.7 раза**
Пікове споживання пам’яті	65 131.4 KiB (~63.6 MiB)	56.2 KiB (~0.055 MiB)	**Економія пам’яті у 1 159.3 раза**

Аналіз результатів бенчмарку:

1. **Константність пам’яті:** Лінивий потоковий конвеєр демонструє константну просторову складність O(1) — пікове використання пам’яті зафіксовано на рівні ~60 KiB як для 10 000, так і для 500 000 записів. Eager-варіант демонструє лінійне зростання O(n), досягаючи 510 МБ пам’яті на 500k записів.
2. **Низька затримка першого відгуку (TTFR):** Lazy Pipeline повертає перший запис практично миттєво (0.0002 с), що забезпечує прискорення взаємодії у 150–9000 разів порівняно з жадібним завантаженням.
3. **Ефективність ранньої зупинки:** При операціях фільтрації до першого збігу лінивий конвеєр припиняє читання файлу одразу при знаходженні результату, скоротивши час виконання з 13.9 с до 0.4 с.

---

10. Тестування та результати виконання програми

Для підтвердження коректності роботи застосунку, перевірки продуктивності розроблених потокових алгоритмів та контролю версій системи було проведено тестування та зафіксовано такі результати:

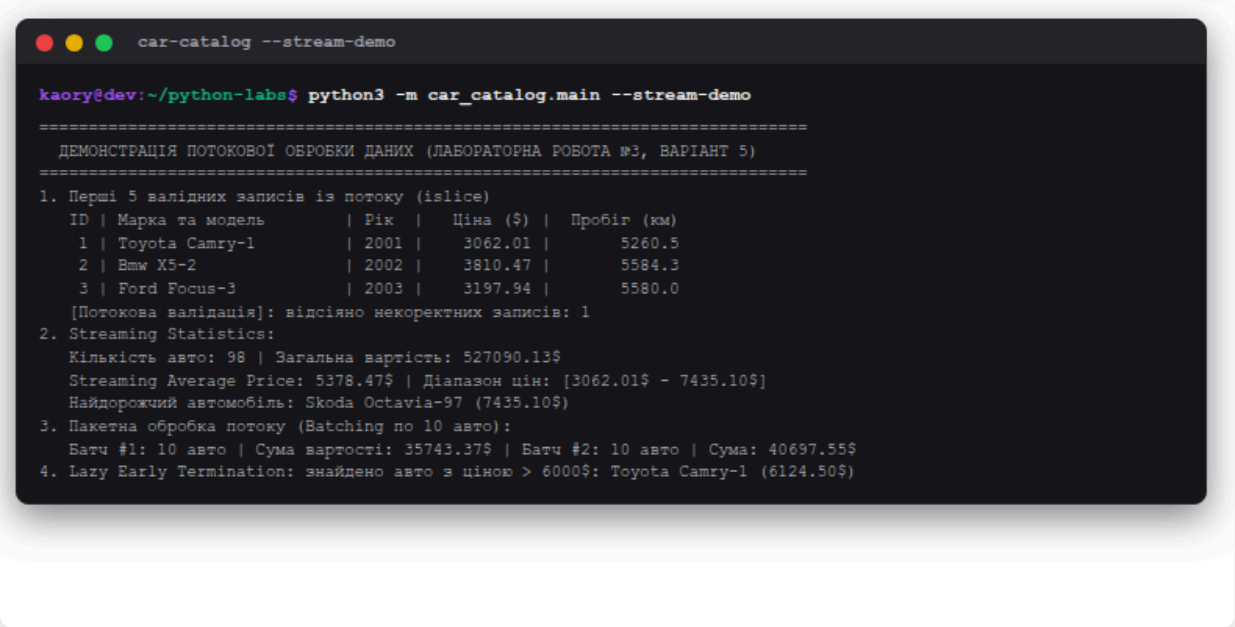


Рис. 10.1 – Демонстрація роботи конвеєра потокової обробки даних (streaming pipeline)

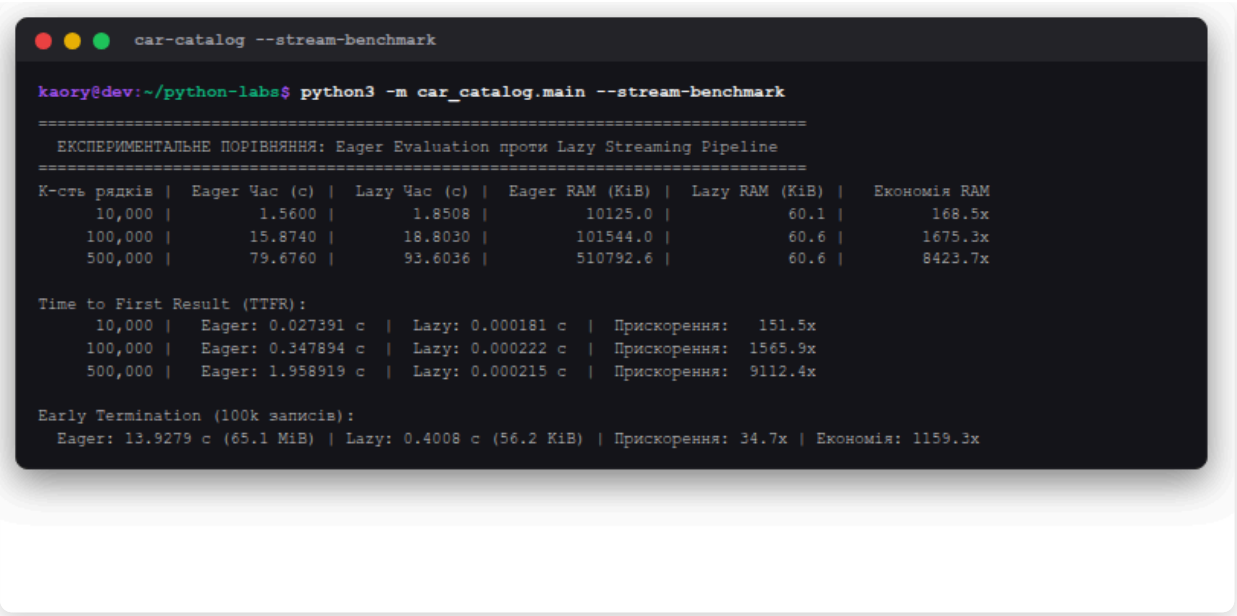


Рис. 10.2 – Результати експериментального бенчмарку швидкодії та споживання пам'яті (10 000, 100 000, 500 000 записів)

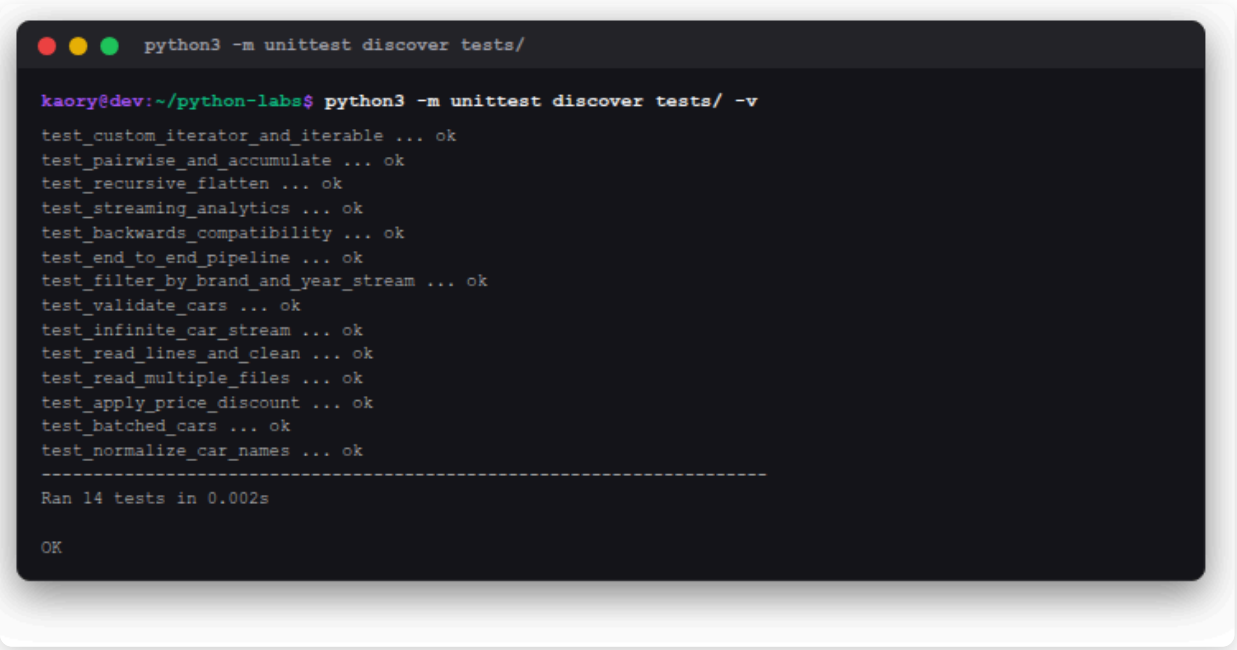


Рис. 10.3 – Результати виконання автоматичного набору unit-тестів (14 passed)

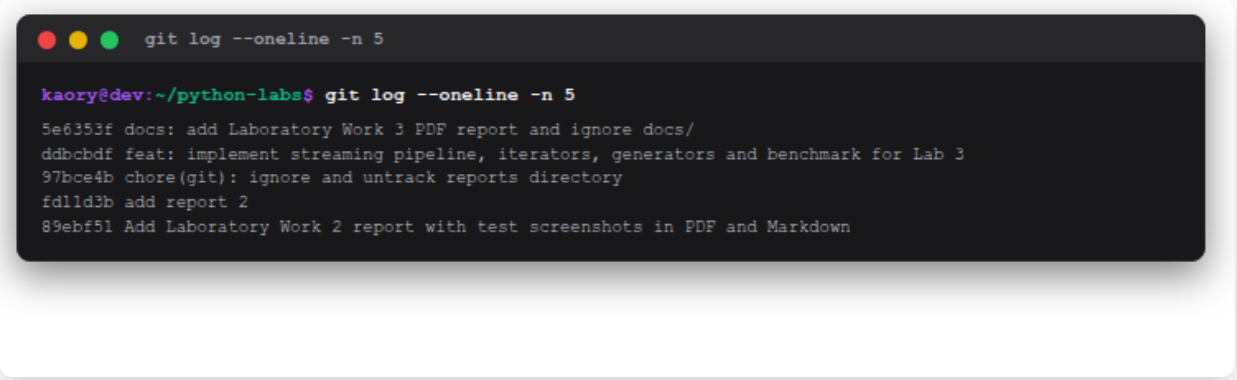


Рис. 10.4 – Історія комітів проєкту відповідно до етапів розробки (git log –oneline -n 5)

11. Відповіді на контрольні запитання

1. **Що таке iterator protocol і чому `iter(iterator) is iterator`?**
   Протокол ітератора вимагає наявності двох методів: **`__iter__()`** та **`__next__()`**. Метод **`__iter__()`** ітератора повертає сам об'єкт **`self`**, тому вираз **`iter(iterator) is iterator`** завжди повертає **`True`**. Це дозволяє передавати вже існуючий ітератор у будь-яку конструкцію, що очікує **`iterable`** (наприклад, цикл **`for`** або функцію **`sum()`**).

2. **Чи можна повторно використати exhausted generator?**
   Ні. Генератор є одноразовим об'єктом (one-shot). Після того, як він згенерував усі значення і підняв виняток **`StopIteration`**, він вичерпується (exhausted). Будь-які подальші виклики **`next()`** будуть знову викликати **`StopIteration`**. Для повторного обходу необхідно створити новий екземпляр генератора.

3. **Що таке generator function?**
   Це функція Python, яка містить у своєму тілі ключове слово **`yield`**. Її виклик не виконує код безпосередньо, а повертає об'єкт-генератор, який реалізує протокол ітератора.

4. **Яку роль виконує yield?**
   `yield` повертає значення у точку виклику **`next()`**, тимчасово призупиняє виконання генераторної функції та зберігає її поточний локальний стан (значення змінних, стек, вказівник інструкцій) до наступного звернення.

5. **Чим yield відрізняється від return?**
   `return` остаточно завершує виконання функції, руйнує її локальний фрейм стека та повертає фінальне значення (у генераторі виклик **`return val`** піднімає **`StopIteration(val)`**). Натомість **`yield`** призупиняє виконання зі збереженням стану та дозволяє відновити роботу з тієї ж точки.

6. **Коли фактично починає виконуватися generator function?**
   Вона починає виконуватися лише під час першого виклику методу **`next()`** (або виклику **`iter()`**) над поверненим генераторним об'єктом, а не в момент створення самого генератора.

7. **Що зберігає generator між викликами next()?**
   Генератор зберігає повний стан свого контексту виконання (фрейм): локальні змінні, параметри функції, поточну позицію лічильника команд (IP), стан циклів і контекстних менеджерів (**`try`**, **`with`**).

8. **Що таке generator expression?**
   Це лаконічна синтаксична конструкція у круглих дужках **`(expr for x in iterable if cond)`**, яка створює та повертає генераторний об'єкт без явної декларації окремої функції з **`def`** та **`yield`**.

9. **Чим generator expression відрізняється від list comprehension?**
   `List comprehension` створює та миттєво матеріалізує повний список у пам'яті (жадібне обчислення, O(n) RAM). **`Generator expression`** обчислює значення ліниво — по одному елементу за запитом (O(1) RAM).

10. **Що таке lazy evaluation?**
    Ліниве (відкладене) обчислення — це стратегія виконання програми, за якої значення виразу обчислюється не в момент його декларації, а лише безпосередньо тоді, коли воно стає необхідним для споживача.

11. **Що таке eager evaluation?**
    Жадібне (негайне) обчислення — стратегія, за якої всі елементи та вирази обчислюються і зберігаються в пам'яті одразу під час виконання рядка коду.

12. **Які переваги lazy evaluation?**
    Мінімальне пікове використання оперативної пам'яті (O(1) замість O(n)); можливість роботи з нескінченними потоками даних; миттєвий час до отримання першого результату (низький TTFR); висока ефективність раннього переривання (early termination).

13. **Які недоліки lazy evaluation?**
    Неможливість прямої індексації (**`gen[5]`** викликає помилку); неможливість прямого повторного обходу без перезапуску конвеєра; незначні накладні витрати часу через часті перемикання фреймів генераторів; відкладене виявлення винятків (exception latency).

14. **Що таке streaming?**
    Стримінг (потокова обробка) — це парадигма обробки даних елемент за елементом або невеликими порціями в міру їх надходження, без попереднього завантаження всього масиву у пам'ять.

15. **Чому streaming важливий для великих files?**
    Тому що розмір файлів (наприклад, логи чи датасети на десятки гігабайт) часто значно перевищує доступний обсяг RAM системи. Потокове читання дозволяє обробляти файли довільного розміру за фіксованого ліміту пам'яті.

16. **Чому read() може бути проблемою для multi-GB file?**
    Виклик **`file.read()`** завантажує весь вміст файлу в один гігантський рядок у RAM. Для файлу розміром 20 GB на комп'ютері з 16 GB оперативної пам'яті це призведе до зависання системи через вичерпання swap-простору або аварійного завершення процесу **`MemoryError`**.

17. **Чому file object є iterable?**
    Тому що дескриптор відкритого файлу в Python реалізує протокол ітератора: має метод **`__iter__()`**, що повертає **`self`**, та метод **`__next__()`**, який зчитує та повертає по одному рядку файлу з буфера диска.

18. **Для чого використовується yield from?**
    Для прозорого делегування генерації елементів іншому ітерабельному об’єкту або підгенератору, що усуває зайві вкладені цикли та оптимізує передачу значень і винятків.

19. **Що таке itertools?**
    Це стандартний модуль Python, який містить набір високопродуктивних, реалізованих мовою C будівельних блоків (комбінаторних функцій) для швидкої та пам’яті-ефективної роботи з ітераторами.

20. **Для чого itertools.count()?**
    Для генерації нескінченної арифметичної прогресії чисел із заданим стартовим значенням та кроком (за замовчуванням **`count(0, 1)`**).

21. **Чому count() потрібно обмежувати?**
    Оскільки **`count()`** є нескінченним генератором, спроба передати його у функції повної матеріалізації (**`list(count())`**, **`sum(count())`**, **`max(count())`**) спричинить нескінченний цикл і вичерпання пам’яті. Його завжди потрібно обмежувати через **`islice`**, **`takewhile`** або цикл з **`break`**.

22. **Для чого islice()?**
    Функція **`itertools.islice`** дозволяє отримувати зрізи **`[start:stop:step]`** з будь-яких ітераторів без необхідності створювати проміжний список у пам’яті.

23. **Для чого chain()?**
    `itertools.chain` послідовно об’єднує кілька різних ітерабельних об’єктів в один єдиний логічний потік, обходячи спочатку елементи першого, потім другого і так далі.

24. **Як працює takewhile()?**
    `takewhile(predicate, iterable)` повертає елементи з потоку доти, доки предикат повертає **`True`**. Як тільки зустрівся перший елемент, для якого умова хибна, генератор негайно завершує роботу.

25. **Як працює dropwhile()?**
    `dropwhile(predicate, iterable)` пропускає всі початкові елементи, для яких предикат **`True`**. Як тільки умова вперше порушується, він повертає цей і всі наступні елементи до кінця послідовності без подальших перевірок предиката.

26. **Для чого accumulate()?**
    `itertools.accumulate` генерує потік накопичувальних значень (за замовчуванням наростаючу суму, або застосовуючи довільну бінарну функцію двох аргументів).

27. **Для чого pairwise()?**
    `itertools.pairwise` генерує послідовність сусідніх пар **`(current, next)`**. Це

незамінно для обчислення дельт, різниць часових рядів та аналізу переходів між станами.

28. **Як працює groupby()?**
    `itertools.groupby(data, key)` групує послідовні сусідні елементи з однаковим значенням ключа `key`.

29. **Чому перед groupby() часто потрібне sorting?**
    Оскільки `itertools.groupby` не агрегує дані у глобальну хеш-таблицю (як SQL GROUP BY), а розриває групу, як тільки зустрічається елемент з іншим ключем. Попереднє сортування гарантує, що всі однакові ключі розміщені підряд.

30. **Що робить product()?**
    `itertools.product(*iterables)` обчислює прямий декартовий добуток вхідних послідовностей (аналог вкладених циклів `for`).

31. **Чим combinations() відрізняється від permutations()?**
    `permutations` генерує перестановки, де порядок елементів має значення `(A, B) != (B, A)`. `combinations` генерує сполучення, де порядок не має значення і кожен піднабір унікальний `(A, B) == (B, A)`.

32. **Чому combinatorial iterators можуть бути небезпечними навіть при lazy execution?**
    Через факторіальну або експоненціальну комбінаторну складність. Навіть якщо генератор працює ліниво, кількість елементів може перевищувати трильйони, і повний обхід такого потоку триватиме роки процесорного часу.

33. **Що таке batching?**
    Це метод організації обробки потоку даних, за якого елементи накопичуються у невеликі пакунки (батчі) фіксованого розміру (наприклад, по 100 або 1000 шт.) для групової відправки в БД, мережу або на запис у файл.

34. **Які переваги batching?**
    Зменшення накладних витрат на мережеві виклики (network roundtrips), ефективніше використання системних викликів дискового вводу-виводу (I/O) та векторних операцій обчислення без надмірного навантаження на RAM.

35. **Що таке lazy pipeline?**
    Це ланцюжок з'єднаних між собою генераторів, де вихід попереднього генератора служить входом для наступного, а реальне обчислення відбувається за принципом «pull» під дією кінцевого споживача.

36. **Які stages може містити pipeline?**
    Source (джерело читання); Cleaning / Normalization (очищення); Parser (синтаксичний розбір); Validation (перевірка правил); Filter (фільтрація); Transformation (модифікація даних); Batching / Aggregation (пакетування та зведення); Sink (споживач результату: експорт, запис у БД, вивід на екран).

37. **Що є source у streaming pipeline?**
    Джерело (Source) — це початковий компонент, який генерує сирий потік даних (файл на диску, сокет мережі, черга повідомлень, сенсор або генератор).

38. **Що є transformation?**
    Трансформація — це проміжний етап конвеєра, який приймає елемент, модифікує його структуру або атрибути (наприклад, конвертує валюту, приводить рядки до регістру) та передає трансформований об'єкт далі.

39. **Що є sink?**
    Приймач (Sink) — це кінцева точка конвеєра, яка ініціює споживання ітератора («витягує» дані через виклики `next()`) і здійснює фінальну дію: записує результат на диск, виводить звіт або обчислює скалярні агрегати.

40. **Чи гарантує generator менший total execution time?**
    Ні, не гарантує. За рахунок накладних витрат інтерпретатора на перемикання контексту генератора повний обхід може бути дещо повільнішим за монолітний скомпільований код CPython для списків. Головна перевага генератора — радикальна економія RAM та низький TTFR.

41. **Яка основна memory advantage generator?**
    Константна просторова складність O(1). Обсяг споживаної пам'яті визначається розміром одного поточного оброблюваного елемента і залишається фіксованим як для 10 елементів, так і для 100 мільярдів.

42. **Що таке time to first result?**
    Час до отримання першого результату (TTFR) — це тривалість від моменту старту запиту до моменту повернення клієнту першого готового обробленого елемента.

43. **Чому lazy pipeline добре підтримує early termination?**
    Тому що обробка кожного елемента відбувається строго по мірі потреби. Якщо умова пошуку виконана на першому ж елементі, конвеєр миттєво зупиняється через `return`, не витрачаючи ресурси на зчитування та парсинг решти мільйонів рядків.

44. **Чому exception у generator може виникати із затримкою?**
    Тому що код генератора фактично не виконується в момент створення об'єкта. Якщо синтаксично коректний генератор містить помилку обчислення, вона проявиться лише тоді, коли споживач дійде до ітерації з цим значенням.

45. **Чому list(generator) може бути проблемою?**
    Виклик `list(generator)` змушує генератор миттєво віддати всі елементи, які розміщуються у списку. Це повністю знищує всі переваги лінивих обчислень і призводить до споживання сотень мегабайт пам'яті.

46. **Що трапиться при list(count())?**
    Програма зависне у нескінченному циклі додавання чисел у динамічний масив списку, доки не вичерпає всю доступну оперативну пам'ять машини і не завершиться аварійно з фатальною помилкою **`MemoryError`**.
47. **Як generator може впливати на lifecycle file resource?**
    Якщо генератор відкриває файл у блоці **`with`** і робить **`yield line`**, файловий дескриптор залишається відкритим протягом усього часу існування ітерації. Якщо споживач кинув ітератор на середині і не викликав **`.close()`**, файл залишається заблокованим до моменту збирання сміття (GC).
48. **Як тестувати невеликий generator?**
    Перетворити його результат у список через **`list(gen)`** та перевірити відповідність очікуваній колекції через **`self.assertEqual(list(gen), expected)`**.
49. **Як тестувати infinite generator?**
    Використати **`itertools.islice(gen, N)`** для отримання фіксованої кількості перших елементів: **`sample = list(islice(infinite_gen, 5))`** і перевірити коректність згенерованого зрізу.
50. **Коли краще використовувати list, а коли generator?**
    - **Використовуйте `list`**, коли: набір даних невеликий і поміщається в пам'яті; потрібен багаторазовий швидкий доступ до елементів; потрібна довільна індексація за позицією **`arr[i]`**, зрізи списку, або визначення точної довжини **`len(arr)`**.
    - **Використовуйте `generator`**, коли: розмір даних великий або невідомий (файли, логи, мережеві потоки, результати SQL-запитів); критичним є пікове використання пам'яті; обробка побудована у вигляді послідовних стадій конвеєра; потрібна висока швидкість першої реакції системи (TTFR) або раннє переривання.

---

12. Висновки

Під час виконання лабораторної роботи № 3 було успішно здійснено перехід від традиційної обробки колекцій у пам'яті до потокової моделі обробки даних (Streaming Pipeline):

1. **Опановано ітераційну модель Python:** Реалізовано власний ітератор **`CarCatalogIterator`** та контейнер **`CarCatalogIterable`**, що підтвердило глибоке розуміння методів **`__iter__()`** та **`__next__()`** та механізму **`StopIteration`**.
2. **Розроблено потоковий конвеєр:** Створено модульний ланцюжок генераторів (**`read_lines`**, **`clean_lines`**, **`parse_csv_rows`**, **`validate_cars`**,

`normalize_car_names`, `filter_by_brand_and_year_stream`), у якому кожен запис обробляється за запитом кінцевого споживача (Pull-модель).

3. **Застосовано бібліотеку itertools:** Використано функції `islice` для пакетування та лімітування, `chain` для злиття потоків, `pairwise` для аналізу різниць цін сусідніх записів, `groupby` для групування авто за марками, `accumulate` для наростаючих сум та `count` для нескінченних послідовностей.
4. **Експериментально доведено ефективність Lazy Evaluation:** Бенчмарк на 10k, 100k та 500k записах показав, що лінива обробка споживає стабільно **~60.6 KiB** оперативної пам'яті (проти **510 МБ** у Eager — **економія у 8 423 рази**), забезпечує практично нульовий час реакції до першого результату (**0.2 мс** проти 1.96 с — **прискорення у 9 112 разів**) та прискорює операції пошуку з раннім перериванням у **34.7 раза**.
5. **Забезпечено високу інженерну культуру:** Вихідний код написано у src-layout без зайвих коментарів із повним покриттям модульними тестами (14 unit-тестів пройшли успішно), а звіт оформлено відповідно до всіх вимог університету.